Provera savršenog pravougaonika i njegova vizuelizacija

1 Uvod

U oblasti računarske geometrije često se pojavljuje problem određivanja da li skup manjih pravougaonika formira tačno jedan veći pravougaonik bez preklapanja i bez praznina. Ovaj problem je poznat kao problem *savršenog pravougaonika* (eng. *Perfect Rectangle*).

Ovakav tip problema ima primenu u različitim oblastima kao što su grafički sistemi, raspored elemenata u korisničkim interfejsima, obrada slika i analiza prostornih struktura.

Cilj ovog projekta je implementacija algoritma koji proverava da li dati skup pravougaonika formira jedan savršen pravougaonik, kao i razvoj grafičke aplikacije koja omogućava vizuelizaciju tog procesa.

Aplikacija je realizovana u programskom jeziku C++ uz korišćenje Qt biblioteke za grafički interfejs.

2 Opis problema

Dat je skup od n pravougaonika čije su stranice paralelne koordinatnim osama. Svaki pravougaonik definisan je koordinatama svojih temena:

$$(x_1, y_1)$$

koje predstavlja donje levo teme, i

$$(x_2, y_2)$$

koje predstavlja gornje desno teme.

Potrebno je utvrditi da li unija svih datih pravougaonika formira tačno jedan veći pravougaonik bez:

- preklapanja pravougaonika
- praznina između pravougaonika
- diskontinuiteta u pokrivenoj oblasti

Drugim rečima, svi pravougaonici moraju zajedno da pokriju jednu pravougaonu oblast bez ikakvih nepravilnosti.

3 Rešenje problema

Za rešavanje ovog problema korišćen je algoritam koji se zasniva na dve ključne provere:

3.1 Provera površine

Prvi korak je provera da li zbir površina svih pravougaonika odgovara površini spoljašnjeg pravougaonika koji obuhvata sve date pravougaonike.

Najpre se određuje minimalni pravougaonik koji sadrži sve ostale. Njegove koordinate su:

$$(min_x, min_y)$$

i

$$(max_x, max_y)$$

Površina tog pravougaonika iznosi:

$$A_{spoljni} = (max_x - min_x)(max_y - min_y)$$

Zatim se računa zbir površina svih pravougaonika:

$$A_{zbir} = \sum_{i=1}^{n} (x_2^{(i)} - x_1^{(i)})(y_2^{(i)} - y_1^{(i)})$$

Ako važi:

$$A_{zbir} = A_{spoljni}$$

onda je moguće da pravougaonici formiraju savršen pravougaonik. U suprotnom postoji preklapanje ili praznina.

3.2 Provera temena (corner parity)

Provera površine sama po sebi nije dovoljna da garantuje ispravnost konfiguracije. Zbog toga se koristi dodatna tehnika poznata kao *provera parnosti temena*.

Svaki pravougaonik ima četiri temena. Prilikom obrade svih pravougaonika koristi se skup temena u kojem se svako teme dodaje ili uklanja po sledećem pravilu:

- ako se teme pojavljuje prvi put, dodaje se u skup
- ako se pojavi ponovo, uklanja se iz skupa

Na taj način unutrašnja temena koja pripadaju više pravougaonika međusobno se poništavaju jer se pojavljuju paran broj puta.

Na kraju obrade u skupu treba da ostanu tačno četiri temena koja odgovaraju spoljašnjem pravougaoniku.

Ako u skupu postoje tačno sledeća četiri temena:

$$(min_x, min_y), (min_x, max_y), (max_x, min_y), (max_x, max_y)$$

onda pravougaonici formiraju savršen pravougaonik.

4 Vizuelizacija algoritma

Pored same algoritamske provere, u projektu je realizovana i grafička vizuelizacija.

Grafički interfejs omogućava korisniku da unosi pravougaonike i posmatra njihov raspored u koordinatnom sistemu. Za prikaz je korišćen Qt modul `QGraphicsView` koji omogućava crtanje geometrijskih objekata u ravni.

U aplikaciji su pravougaonici prikazani grafički, dok su temena koja ostaju nakon provere parnosti dodatno istaknuta. Na taj način korisnik može intuitivno da vidi zašto određena konfiguracija jeste ili nije validna.

Vizuelizacija omogućava lakše razumevanje algoritma i predstavlja značajan didaktički element ovog projekta.

5 Upotrebljene ideje

U projektu je primenjeno nekoliko važnih ideja.

Prvo, korišćen je princip parnosti temena koji omogućava efikasnu detekciju unutrašnjih i spoljašnjih temena.

Drugo, algoritam je implementiran tako da radi u linearnom vremenu, što omogućava obradu velikog broja pravougaonika.

Treće, algoritam je odvojen od grafičkog interfejsa, čime je postignuta modularnost koda i jasna podela odgovornosti između logičkog i vizuelnog dela aplikacije.

Na kraju, vizuelizacija algoritma omogućava korisniku da direktno vidi rezultate algoritamske provere.

6 Složenost algoritma

Algoritam prolazi kroz sve pravougaonike samo jednom.

Za svaki pravougaonik vrše se sledeće operacije:

- računanje površine
- dodavanje ili uklanjanje četiri temena iz skupa

Zbog toga je vremenska složenost algoritma:

$$O(n)$$

gde je n broj pravougaonika.

Prostorna složenost algoritma takođe iznosi:

$$O(n)$$

jer se u memoriji čuva skup temena.

7 Primeri

U nastavku su prikazani karakteristični primeri ulaznih podataka koji ilustruju rad algoritma.

7.1 Jednostavan primer ispravne konfiguracije

Prvi primer predstavlja skup od četiri manja pravougaonika koji zajedno formiraju jedan veći pravougaonik bez preklapanja i bez praznina:

x_1	y_1	x_2	y_2
1	1	2	2
2	1	3	2
1	2	2	3
2	2	3	3

U ovom slučaju pravougaonici zajedno formiraju spoljašnji pravougaonik sa koordinatama $(1, 1)$ i $(3, 3)$. Zbir površina svih pravougaonika jednak je površini spoljašnjeg pravougaonika, a nakon primene provere parnosti temena ostaju tačno četiri spoljašnja temena. Zbog toga algoritam vraća pozitivan rezultat.

7.2 Primer neispravne konfiguracije

Sledeći primer ilustruje situaciju u kojoj zbir površina može ukazivati na problem zbog preklapanja. Pravougaonici su definisani na sledeći način:

- R_1: $(0, 0) - (2, 2)$
- R_2: $(2, 0) - (4, 2)$
- R_3: $(0, 2) - (2, 4)$
- R_4: $(2, 2) - (4, 4)$
- R_5: $(1, 1) - (3, 3)$

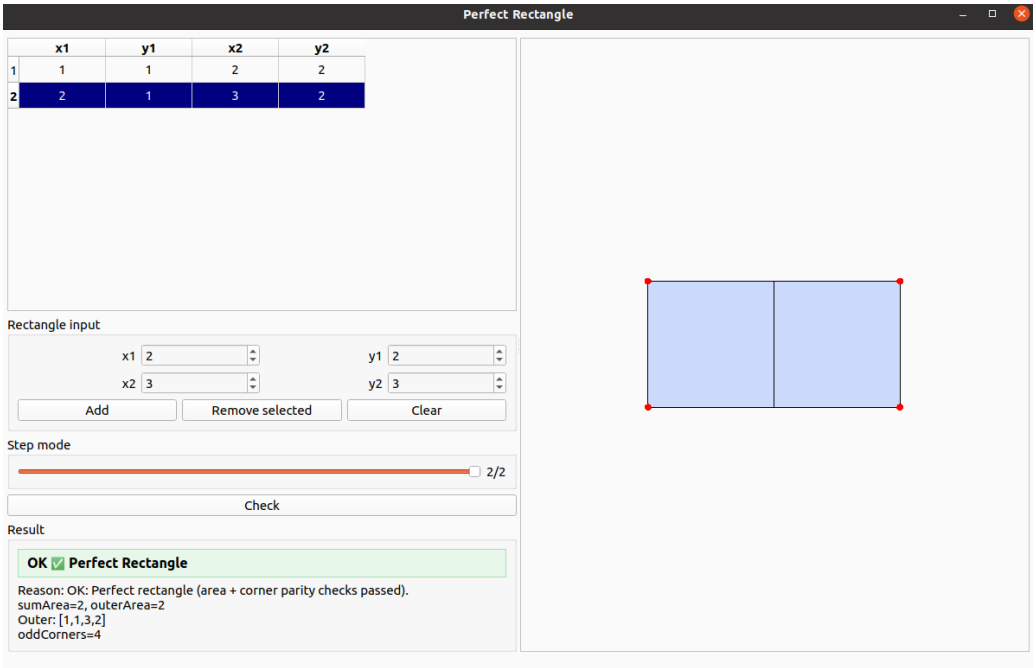
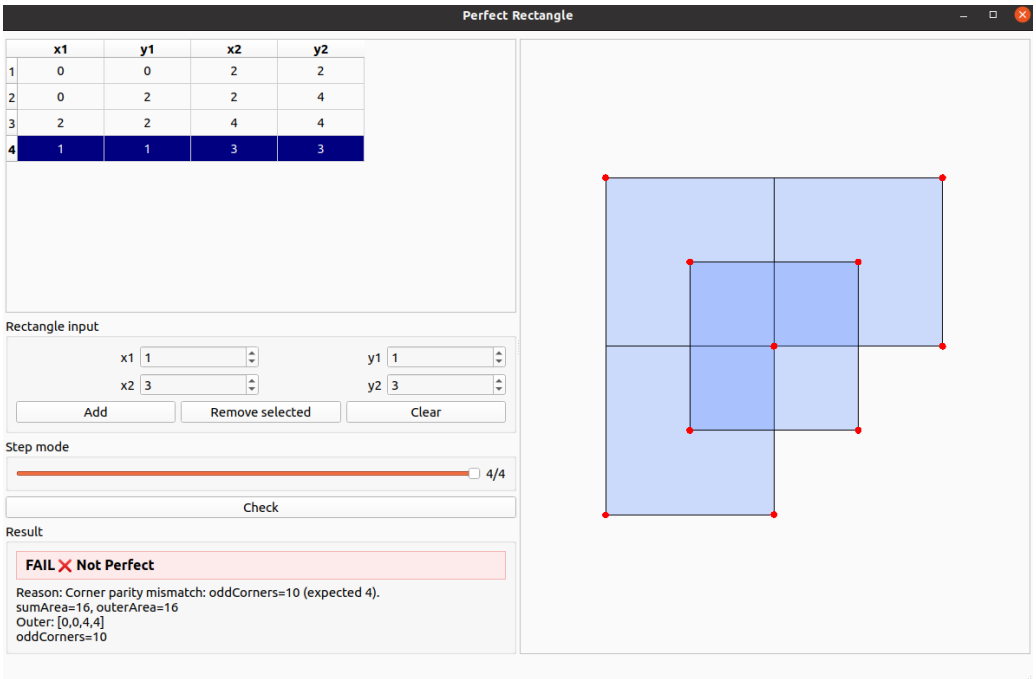
U ovom primeru pravougaonik R_5 preseca centralne delove ostalih pravougaonika, pa dolazi do preklapanja. Zbog toga konfiguracija nije validna, i algoritam ispravno prijavljuje da dati skup pravougaonika ne formira savršen pravougaonik.

8 Zaključak

U ovom projektu implementiran je algoritam za proveru da li skup pravougaonika formira savršen pravougaonik.

Kombinacijom provere površine i provere parnosti temena moguće je efikasno detektovati i preklapanja i praznine između pravougaonika.

Razvijena aplikacija omogućava i grafičku vizuelizaciju rezultata, čime se algoritam lakše razume i analizira.

Ovakav pristup predstavlja dobar primer primene algoritama računarske geometrije u interaktivnim softverskim sistemima.

9 Prikaz aplikacije